

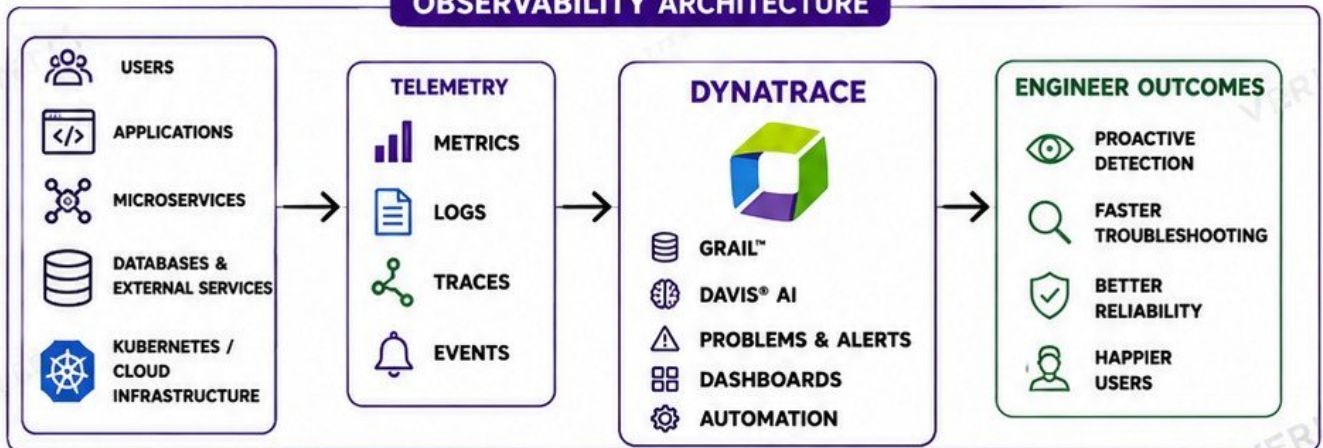
DYNATRACE

PRODUCTION HANDBOOK

**FROM OBSERVABILITY FUNDAMENTALS TO
PRODUCTION MONITORING AND TROUBLESHOOTING**



OBSERVABILITY ARCHITECTURE



WHAT YOU WILL MASTER

- | | | |
|------------------------------|---------------------------------|------------------------|
| ✓ DYNATRACE FUNDAMENTALS | ✓ ALERTING & PROBLEMS | ✓ SYNTHETIC & RUM |
| ✓ ONEAGENT & DISCOVERY | ✓ KUBERNETES & CLOUD MONITORING | ✓ BEST PRACTICES |
| ✓ METRICS, LOGS & TRACES | ✓ SLOS AND RELIABILITY | ✓ PRODUCTION LABS |
| ✓ DQL QUERIES | ✓ SECURITY & ADMINISTRATION | ✓ REAL WORLD SCENARIOS |
| ✓ DASHBOARDS & VISUALIZATION | ✓ INCIDENT TROUBLESHOOTING | ✓ AND MUCH MORE |



OBSERVE EVERYTHING. UNDERSTAND DEEPER. FIX FASTER. SHIP CONFIDENTLY.

DYNATRACE EXPLAINED, WHY ENGINEERS USE IT

THE FOUNDATION OF MODERN OBSERVABILITY

1 WHAT DYNATRACE IS



DynaTrace is an AI-powered observability platform that automatically collects, correlates, and analyzes telemetry from your entire stack in real time.

It gives engineers full visibility across applications, infrastructure, and users to deliver and operate software with confidence.

2 MONITORING vs OBSERVABILITY

MONITORING

- ✗ Tells you WHAT is wrong
- ✗ Focus on predefined metrics
- ✗ Set static thresholds
- ✗ Siloed data (metrics or logs)
- ✗ Reactive: You get alerted
- ✗ Harder to find the root cause

OBSERVABILITY

- ✓ Helps you understand WHY
- ✓ Uses all telemetry signals
- ✓ AI-driven, context aware
- ✓ Correlated, full-stack view
- ✓ Proactive: Detects issues early
- ✓ Faster root cause analysis

3 THE PRODUCTION VISIBILITY PROBLEM

- ⚠ Complex distributed systems
- ⚠ Many components, many teams
- ⚠ High traffic and user expectations
- ⚠ Issues are hidden across layers
- ⚠ Too much data, not enough answers
- ⚠ MTTR is slow, customers are impacted

**ENGINEERS NEED CLARITY,
NOT MORE NOISE.**

4 THE FIVE PILLARS OF TELEMETRY



METRICS

Numbers over time. Used for trends, capacity planning, and alerting.



LOGS

Detailed records of events. Used for debugging and auditing.



TRACES

Request journeys across services. Used to find latency and errors.



EVENTS

Important changes or anomalies in the system. (Deployments, failures, etc.)



CONTEXT

Metadata that connects everything (entities, relationships, user experience).

5 FULL-STACK OBSERVABILITY



APPLICATIONS

APIs, microservices, runtimes, transactions, user experience



INFRASTRUCTURE

Hosts, containers, Kubernetes, networks, OS, cloud



DEPENDENCIES

Databases, caches, queues, external services



USERS

Real user monitoring, synthetic monitoring



BUSINESS CONTEXT

Service levels, business metrics, reliability, performance

6 WHERE DYNATRACE FITS



FOR DEVOPS

- See the impact of every change
- Detect issues before users report
- Speed up releases with confidence
- Reduce Mean Time to Recovery (MTTR)



FOR SRE

- Maintain reliability and SLAs
- Proactive problem detection
- Understand system behavior
- Optimize performance and capacity



FOR PLATFORM ENGINEERS

- Provide visibility as a platform capability
- Standardize observability across teams
- Reduce tool sprawl and manual effort

7 MENTAL MODEL



APPLICATION

Your system runs and serves users



TELEMETRY

Metrics, logs, traces, events are generated



DYNATRACE

Collects, correlates, analyzes with AI



ENGINEER

Understands what happened and what to do next

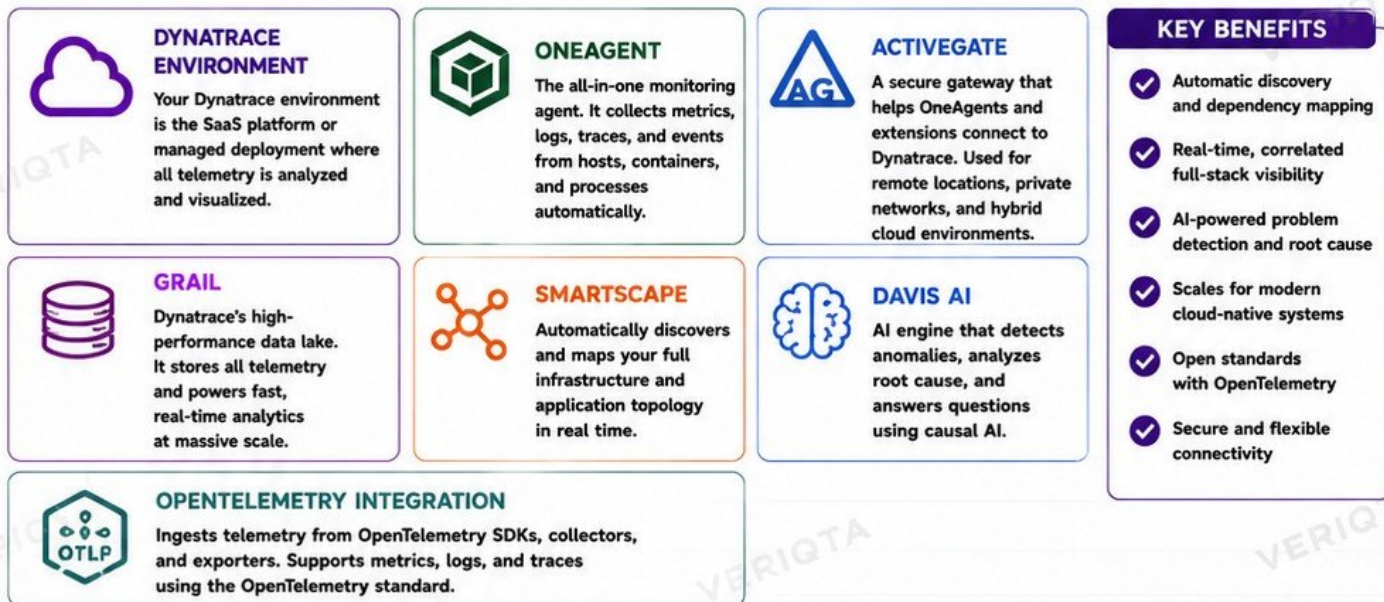


JUNIOR ENGINEER TIP

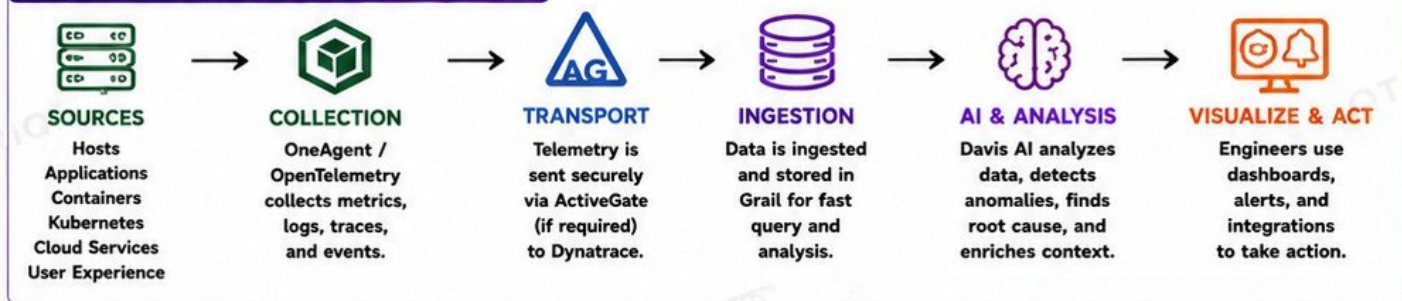
DynaTrace does not replace your engineering skills. It amplifies them. Understand your system first, then use DynaTrace to gain clarity faster.

DYNATRACE ARCHITECTURE AND CORE COMPONENTS

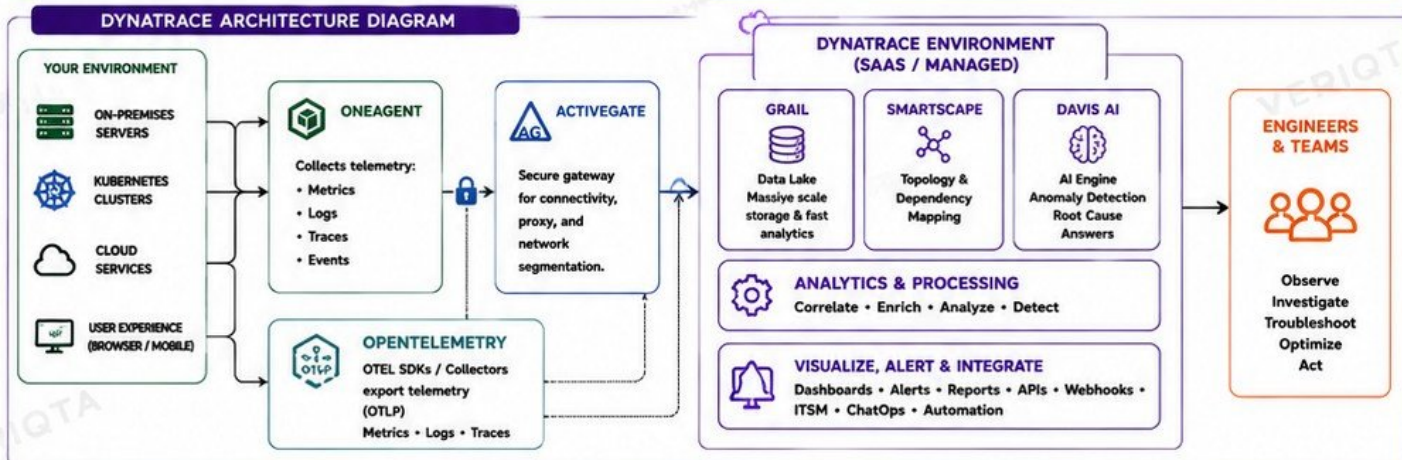
THE BUILDING BLOCKS OF FULL-STACK OBSERVABILITY



HOW TELEMETRY REACHES DYNATRACE



DYNATRACE ARCHITECTURE DIAGRAM



JUNIOR ENGINEER TIP

Understand your system first. Dynatrace gives you the visibility. Your engineering skills turn that visibility into action.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

ONEAGENT AND AUTOMATIC DISCOVERY

ONEAGENT IS THE FOUNDATION OF FULL-STACK OBSERVABILITY

1 WHAT ONEAGENT DOES



OneAgent is a lightweight monitoring agent that automatically collects telemetry, discovers your environment, and provides deep visibility without manual configuration.

- ✓ Collects metrics, logs, traces, and events
- ✓ Automatically discovers and maps infrastructure and dependencies
- ✓ Provides real-time, continuous observability
- ✓ Works across cloud, on-prem, containers, VMs, and Kubernetes
- ✓ No code changes required
- ✓ Auto-instrumentation for supported technologies
- ✓ Sends data securely to Dynatrace
- ✓ Built for modern DevOps and SRE teams

2 HOST MONITORING



- CPU, memory, disk, network and more
- Host availability and performance
- System resource saturation
- Processes and services running on the host
- Kubernetes node monitoring (when applicable)

3 PROCESS DISCOVERY



- Automatically detects all processes
- Groups related processes into process groups
- Tracks resource usage per process
- Detects process health issues and crashes
- No manual configuration

4 SERVICE DISCOVERY



- Discovers all running services and endpoints
- Maps real-time dependencies
- Identifies communication between services
- Shows upstream and downstream impact
- Updates automatically as environment changes

5 APPLICATION VISIBILITY



- Auto-detects applications and technologies
- Tracks requests, response time, errors, throughput
- Shows code-level visibility (methods, classes, SQL, etc.)
- End-to-end transaction visibility
- Business transaction monitoring

6 AUTOMATIC INSTRUMENTATION



- Instruments supported technologies automatically
- No code changes
- Captures traces, metrics and logs out of the box
- Supports Java, .NET, Node.js, Python, Go, PHP, and more
- Works with frameworks and libraries

7 DEPLOYMENT CONCEPTS



DEPLOY ANYWHERE

On-premises, virtual machines, bare metal, containers, Kubernetes, and cloud platforms.



AUTOMATIC REGISTRATION

OneAgent automatically registers to Dynatrace using the environment URL and token.



SECURE CONNECTION

Data is encrypted in transit using TLS. OneAgent initiates outbound connection.



CENTRAL MANAGEMENT

Manage agents, settings, and updates from the Dynatrace environment.



AUTO-UPDATE

OneAgent updates automatically to ensure you always have the latest capabilities.

8 AGENT HEALTH



- OneAgent running: Check status on the host
- Connected: Verify connection to Dynatrace
- Data ingesting: Telemetry flowing correctly
- No errors: Check for agent warnings or failures
- Version: Ensure up-to-date version
- ActiveGate (if used): Verify connectivity and routing
- Use Dynatrace > Manage > OneAgents for details

9 WHAT TO VERIFY AFTER INSTALLATION

- OneAgent is running on the host or container
- Agent shows as "Connected" in Dynatrace
- Host appears in the Infrastructure view
- Processes are discovered and reporting data
- Services are discovered and mapped
- Metrics, logs, and traces are incoming
- No critical problems or data gaps



JUNIOR ENGINEER TIP

OneAgent does the heavy lifting. Install it, verify it, and let it discover your environment. Focus on understanding the data, not on manual configuration.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



OBSERVE EVERYTHING. UNDERSTAND DEEPER. FIX FASTER. SHIP CONFIDENTLY.

UNDERSTANDING THE DYNATRACE ENTITY MODEL

EVERYTHING IN DYNATRACE IS AN ENTITY AND EVERYTHING IS CONNECTED

1 THE CORE ENTITIES

HOSTS



Physical or virtual machines, containers, or cloud instances that run workloads.

PROCESSES & PROCESS GROUPS



Processes are applications running on a host. Process groups combine similar processes for better visibility.

SERVICES



A technology-aware unit of functionality that handles requests and provides value to users or other services.

APPLICATIONS



A collection of services that deliver a business function or end-user experience.

KUBERNETES ENTITIES



Clusters, namespaces, nodes, workloads, pods, containers, and K8s services discovered automatically.

RELATIONSHIPS BETWEEN ENTITIES



Entities are automatically connected based on how they communicate and depend on each other.

DEPENDENCIES



Dependencies show the call flow and reliance between services, databases, external services, and more.

WHY RELATIONSHIPS MATTER IN INCIDENTS



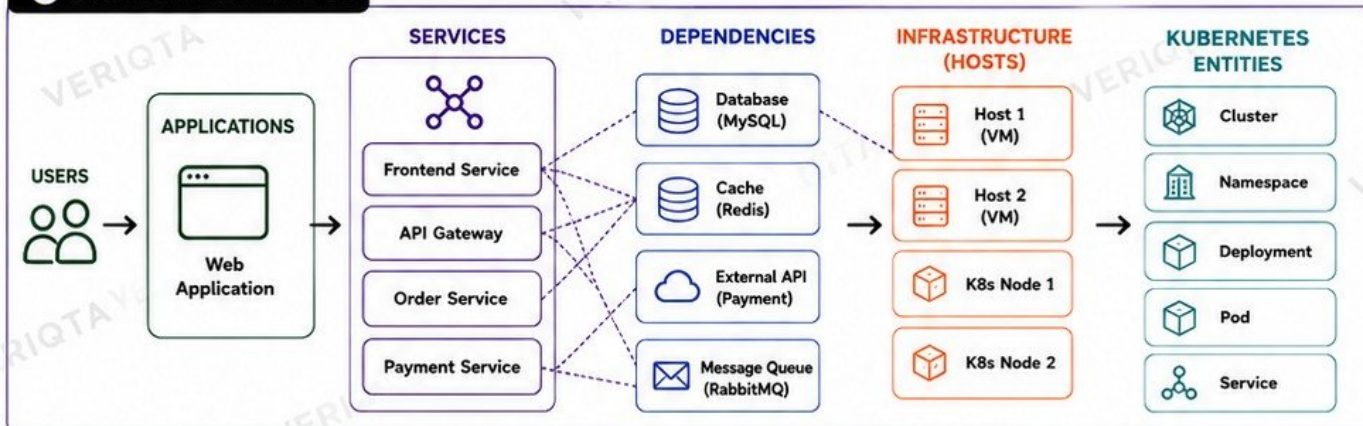
They help you quickly find the root cause by showing what is impacted, what is causing the issue, and what might be affected next.

SMARTSCAPE MENTAL MODEL



Dynatrace's live map of your entire environment. It shows all entities and how they are connected in real time.

2 SMARTSCAPE EXAMPLE



3 KEY TAKEAWAYS

- ✓ Everything is an entity.
- ✓ Entities are discovered automatically.
- ✓ Entities are connected through relationships.
- ✓ Dependencies show how your system works.
- ✓ SmartScape gives you the big picture in real time.
- ✓ Use the entity model to navigate faster during incidents.

4 JUNIOR ENGINEER TIP



Always start your investigation from the entity experiencing the problem and follow the relationships outward. This saves time and finds the real root cause faster.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



OBSERVE EVERYTHING. UNDERSTAND DEEPER. FIX FASTER. SHIP CONFIDENTLY.

METRICS, LOGS, TRACES, AND EVENTS

THE FOUR TELEMETRY SIGNALS THAT TELL THE COMPLETE STORY

1 WHAT EACH TELEMETRY SIGNAL TELLS YOU

METRICS



- Numerical measurements collected over time.
- Best for spotting trends, patterns, and anomalies.
- Answer: "What is happening?"

LOGS



- Detailed, timestamped records of what happened.
- Best for deep investigation and root cause evidence.
- Answer: "What exactly happened?"

TRACES



- End-to-end request journeys across services.
- Best for understanding latency, flow, and failures.
- Answer: "Where did it happen?"

EVENTS



- Significant state changes in your environment.
- Best for understanding changes and context.
- Answer: "What changed and when?"

2 HOW EACH SIGNAL IS USED

	METRICS	LOGS	TRACES	EVENTS
BEST FOR	Trends, capacity, performance, anomalies	Debugging, errors, exceptions, details	Request flow, latency breakdown, dependencies	Deployments, incidents, changes, configurations
EXAMPLES	CPU usage, response time, error rate, throughput	Error stack traces, exception messages, application logs	HTTP requests, DB calls, service-to-service calls	Deployment started, pod restarted, config changed
TIME VIEW	Aggregated over time (seconds → days)	Point-in-time details (precise timestamp)	Per request / transaction journey	At the moment of change/event
VOLUME	High volume, low detail	Very high volume, high detail	Lower volume, high context	Low volume, high importance

3 CORRELATING TELEMETRY



METRICS

Detect the symptom



LOGS

Find the exact error



TRACES

Follow the request and find the cause



EVENTS

Understand the change/context



CORRELATION IS POWER

Connect metrics, logs, traces, and events using time, entity relationships, and context to find the root cause faster.

4 CHOOSING THE RIGHT SIGNAL DURING TROUBLESHOOTING

	IF YOU NEED TO...	USE THIS SIGNAL
	See overall trends or detect an anomaly	METRICS
	Get details about an error or exception	LOGS
	Understand where time is spent in a request	TRACES
	Know what changed and when	EVENTS
	Combine all signals to find the root cause	ALL OF THEM

5 JUNIOR ENGINEER TIP



DON'T GUESS. OBSERVE.

Start broad with metrics, zoom in with traces, confirm with logs, and understand the context with events. Always use all four signals together.

GOLDEN RULE

Metrics show you there is a problem.
Traces show you where it is.
Logs show you why it happened.
Events show you what changed.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



OBSERVE EVERYTHING. UNDERSTAND DEEPER. FIX FASTER. SHIP CONFIDENTLY.

INFRASTRUCTURE AND HOST MONITORING

SEE EVERYTHING. UNDERSTAND EVERYTHING. KEEP SYSTEMS HEALTHY.

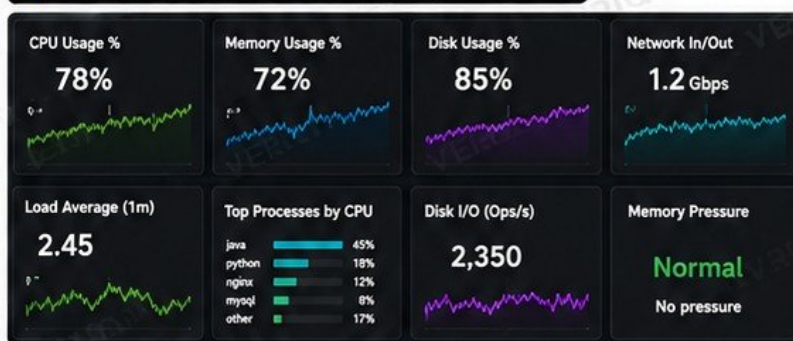
1 CORE INFRASTRUCTURE METRICS

CPU	MEMORY	DISK	NETWORK	PROCESSES	HOST AVAILABILITY
 - CPU utilization - CPU load - Steal time - CPU saturation	 - Memory usage - Used vs available - Swap usage - Memory pressure	 - Disk utilization - I/O read/write - Disk latency - Inodes usage	 - Network throughput - Packets in/out - Errors / drops - Network saturation	 - Top CPU consumers - Top memory consumers - Process count - Zombie processes	 - Host up / down - Agent connectivity - System reboot events - Availability status

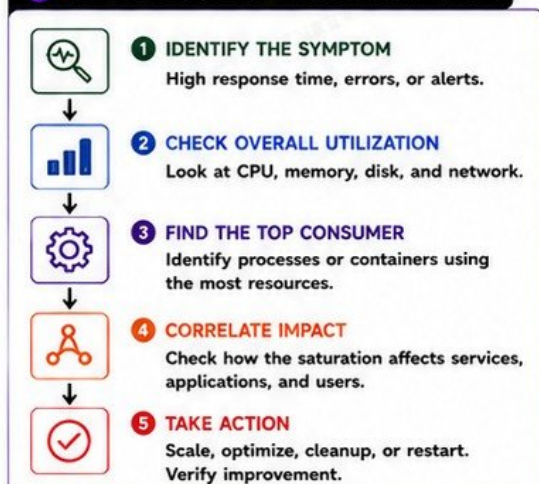
2 RESOURCE SATURATION LEVELS

UTILIZATION	STATUS	WHAT IT MEANS	ACTION
0% - 60%	HEALTHY	Plenty of headroom.	Continue monitoring.
60% - 80%	ELEVATED	Resources are increasing.	Watch trends closely.
80% - 90%	HIGH	At risk of saturation.	Investigate and prepare to act.
90%+	CRITICAL	Saturation likely. Performance impact or outage possible.	Take immediate action.

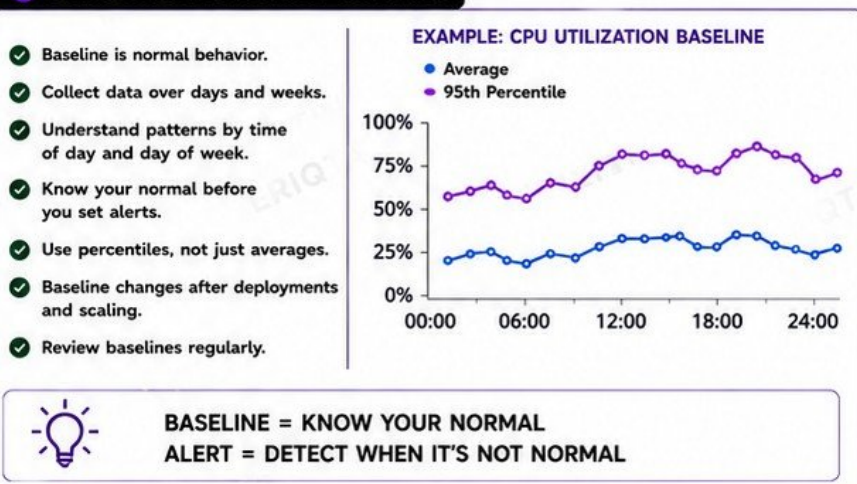
3 INFRASTRUCTURE DASHBOARDS (EXAMPLES)



4 DIAGNOSING AN OVERLOADED HOST



5 PRODUCTION BASELINE THINKING



6 JUNIOR ENGINEER TIP

Always start infrastructure troubleshooting from the host. If the host is unhealthy, everything on it will suffer. Understand the resources, the consumers, the impact, and the trend. That is how you protect production.

KEY QUESTIONS TO ASK

- Is the host reachable?
- Are resources saturated?
- Which process is the top consumer?
- Is this a spike or a trend?
- What is normal for this host?



APPLICATION AND SERVICE MONITORING

SEE WHAT MATTERS. UNDERSTAND WHAT'S SLOW. FIX WHAT HURTS.



1. SERVICES & ENDPOINTS

Services represent the business functions in your application.
#urlpoint individual entry points (URLs, APIs, gRPC methods, etc.) users call.



2. REQUEST RATE

The number of requests received over time.
Helps you understand traffic patterns, spikes, and unusual drops.



3. RESPONSE TIME

The time it takes to respond to a request.
Monitor average, p50, p95, and p99 to catch latency issues early.



4. FAILURE RATE

The percentage of requests that fail.
High failure rate directly impacts users and business reliability.



5. THROUGHPUT

Successful requests processed over time.
Shows how much work your application handles.



6. SERVICE DEPENDENCIES

Services depend on other services, databases, and external systems.
Dependency mapping helps you see the full impact of an issue.



7. DATABASE CALLS

Track database latency, call rate, slow queries, and errors. Databases are a common source of slow performance.



8. EXTERNAL SERVICE CALLS

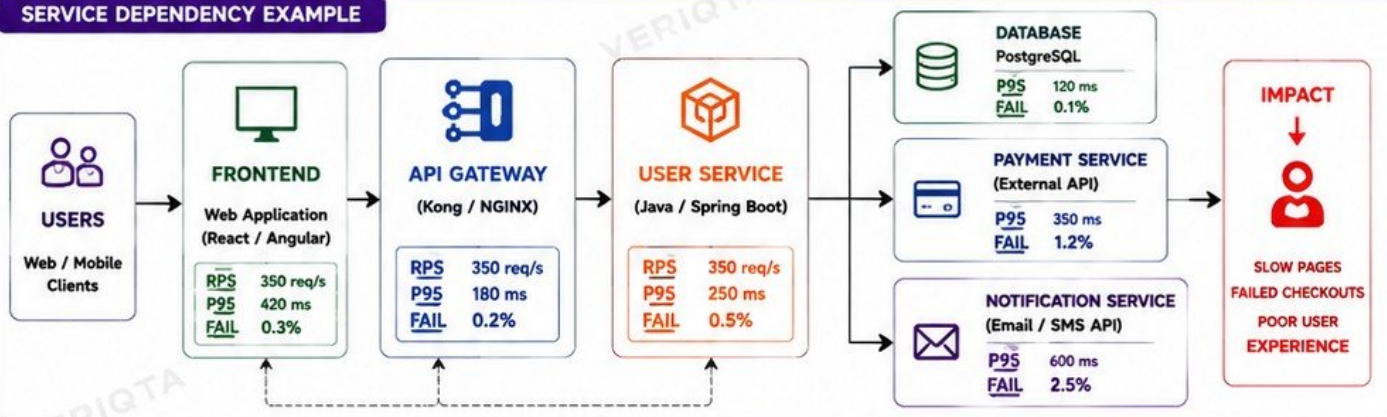
Monitor calls to third-party APIs, SaaS platforms, and external systems. These are outside your control but impact your users.



9. FINDING A SLOW BACKEND SERVICE

Use dependencies, traces, and service breakdowns to identify which service adds the most latency to the user experience.

SERVICE DEPENDENCY EXAMPLE



10. RED METHOD CONNECTION

Dynatrace maps directly to the RED method for service-level monitoring.



RATE

How many requests are coming in?



Dynatrace Metric
Request Rate (RPS)



ERROR

What proportion of requests are failing?



Dynatrace Metric
Failure Rate (%)



DURATION

How long do requests take to complete?



Dynatrace Metric
Response Time (P95/P99)

RED gives you a simple, powerful way to measure reliability and user experience.

HOW ENGINEERS USE THESE SIGNALS

SITUATION	WHAT TO CHECK IN DYNATRACE
Users complain about slowness	→ Response Time (P95/P99) Service Breakdown Traces
Errors increasing	→ Failure Rate Logs Dependency Health
Traffic spike	→ Request Rate Throughput Infrastructure saturation
Slow backend service	→ Dependency Map Service latency Traces (find slow spans)

JUNIOR ENGINEER TIP



Always start from the user impact, then work backward.

Use dependencies and traces to find the real source of latency or failures.

Don't guess.
Follow the data.



GOOD MONITORING IS NOT ABOUT MORE DATA.
IT'S ABOUT THE RIGHT SIGNALS THAT HELP YOU ACT FASTER.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DISTRIBUTED TRACING AND REQUEST ANALYSIS

FOLLOW ONE REQUEST. SEE EVERYTHING. FIND THE REAL BOTTLENECK.

1 WHAT A DISTRIBUTED TRACE REPRESENTS



A distributed trace represents a single request as it travels across multiple services and components.

2 SPANS



A span is a single unit of work within a service. It has a start time, end time, duration, and metadata.

3 PARENT & CHILD RELATIONSHIPS



Spans are organized in a hierarchy. A parent span can have multiple child spans.

4 SERVICE BOUNDARIES



Spans cross service boundaries using context propagation (trace ID).

5 LATENCY CONTRIBUTION



Each span shows how much time it added to the total request duration.

6 ERRORS INSIDE TRACES



Errors are captured inside spans with status codes, error messages, and stack traces for fast troubleshooting.

7 OPENTELEMETRY TRACES



Dynatrace ingests traces from OpenTelemetry (OTLP) and other instrumentation sources.

8 FOLLOWING ONE REQUEST ACROSS MICROSERVICES



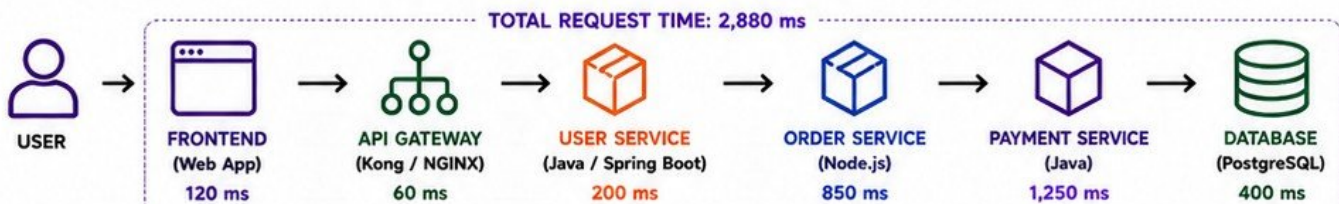
Traces let you follow a single request from the user, through every service, database, and external call, to the final response.

9 FINDING THE ACTUAL BOTTLENECK



By analyzing spans and their durations, you can identify the component that contributes the most latency.

EXAMPLE: A REQUEST FLOW ACROSS MICROSERVICES



TRACE TIMELINE (GANTT VIEW)

SPAN (SERVICE)	DURATION (ms)	TIME (ms)
Frontend (Web App)	120	0 - 120
API Gateway	60	120 - 180
User Service	200	180 - 380
Order Service	850	380 - 1230
Payment Service	1,250	1230 - 2480
Database (PostgreSQL)	400	2480 - 2880
TOTAL	2,880 ms	

HOW TO READ A TRACE

- ✓ Each bar is a span.
- ✓ The position shows when it started.
- ✓ The length shows how long it took.
- ✓ Gaps between bars = waiting time (network, queue, downstream latency).
- ✓ The longest bar is usually your bottleneck.



ERROR EXAMPLE
Span: Payment Service
Status: **ERROR**
Error: Timeout while calling external payment API
Type: timeout
HTTP Code: 504
Duration: 1,250 ms
This error caused the overall request to fail.

FINDING THE BOTTLENECK

- 1 LOOK AT TOTAL TIME**
Start with the total request time.
- 2 COMPARE SPAN DURATIONS**
Find the span with the highest duration.
- 3 CHECK CHILD SPANS**
Drill down into that span to see what it's waiting on.
- 4 VALIDATE THE ROOT CAUSE**
Use logs, metrics, and infrastructure data to confirm.
- 5 FIX AND VERIFY**
Deploy the fix and ensure latency is reduced.

WHY DISTRIBUTED TRACING MATTERS



End-to-end visibility across complex systems



Find the real root cause, not just the symptoms



Reduce Mean Time To Detect (MTTD) and Resolve (MTTR)



Improve user experience and system reliability



Optimize performance and reduce costs



JUNIOR ENGINEER TIP

Traces show you the story of a request. The bottleneck is where the story slows down.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LOG MONITORING AND PRODUCTION INVESTIGATION

LOGS TELL YOU WHAT HAPPENED. CONTEXT TELLS YOU WHY.

1 LOG INGESTION



- Dynatrace ingests logs from many sources.
- OneAgent collects logs from hosts, containers, and applications.
- Use OpenTelemetry (OTLP), API, or integrations.
- Logs are stored in Grail for fast search and analytics.

2 STRUCTURED VS UNSTRUCTURED LOGS



STRUCTURED LOGS

- Key-value or JSON format
- Easy to parse and query
- Searchable by attributes
- Ideal for analytics and alerting



UNSTRUCTURED LOGS

- Free-form text
- Harder to query
- Useful for detailed context and debugging

3 LOG ATTRIBUTES



Attributes add context and make logs useful.

Common attributes:

- timestamp
- log.source
- service.name
- host.name
- container.name
- log.level
- trace.id
- user.id
- environment

4 FILTERING



Use filters to narrow down logs.

- Filter by attributes, values, time range, log level, or text patterns.
- Combine multiple filters for precise results.

5 SEARCHING PRODUCTION LOGS



Use keyword search, attribute filters, and DQL for powerful log searches.

EXAMPLE SEARCHES

Error logs	log.level = "ERROR"
Logs from a service	service.name = "checkout-service"
Logs from a host	host.name = "ip-10-0-1-25"
Specific trace	trace.id = "abc123def456"
Time range	from: now()-30m to: now()
Text search	"OutOfMemoryError"

6 CONNECTING LOGS WITH SERVICES AND TRACES



SERVICE



LOGS



TRACES

- Logs are automatically linked to the service that generated them.
- If trace context exists (trace.id), logs are linked to that trace.
- Follow a trace → open related logs → see the full story of a request.

7 SENSITIVE-DATA CONSIDERATIONS



- Logs can contain sensitive data.
- Never log passwords, tokens, PII, or secrets.
- Use log masking, redaction, or tokenization.
- Apply access controls and least privilege.
- Comply with data privacy regulations (GDPR, CCPA, etc.).

8 TROUBLESHOOTING APPLICATION ERRORS USING LOGS



1. DETECT
Identify the error or issue from metrics, alerts, or users.



2. FIND
Search logs for the error time window and related service.



3. ANALYZE
Read log details, error messages, stack traces, and context.



4. CORRELATE
Correlate with traces, metrics, and events to find the cause.



5. RESOLVE
Fix the issue, deploy, and verify in logs and metrics.

EXAMPLE: INVESTIGATING A 500 ERROR IN CHECKOUT SERVICE

1. SYMPTOM



Users report checkout failures. Alerts show increase in 500 errors.

Service:
checkout-service
Error Rate: 12% ↑

2. SEARCH LOGS

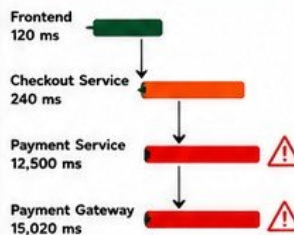
log.level = "ERROR" AND
service.name = "checkout-service"

Results (sample)

```
2025-05-21 10:15:23 ERROR Payment failed  
orderId=9876 userId=4321 error="Timeout"  
trace.id=abc123  
2025-05-21 10:15:24 ERROR Exception in  
PaymentClient call stacktrace=...  
trace.id=abc123  
2025-05-21 10:15:24 ERROR HTTP 504  
from payment-gateway duration=15000ms  
trace.id=abc123
```

3. CORRELATE TRACE

trace.id = abc123



4. FIND ROOT CAUSE



Payment Service calling
external payment gateway
is timing out (15s), causing
500 errors in checkout.

Bottleneck:
Payment Gateway
High latency + Timeouts

5. TAKE ACTION



Fix timeout and
improve retry logic.
Deploy changes and
monitor.

Verify in logs,
metrics, and
traces.



JUNIOR ENGINEER TIP

Logs without context are just noise.
Always filter, correlate, and follow the signals to the root cause.

DYNATRACE QUERY LANGUAGE DQL FUNDAMENTALS

ASK BETTER QUESTIONS. GET CLEARER ANSWERS.

1 WHAT IS DQL?



Dynatrace Query Language (DQL) is a powerful, SQL-like language used to query telemetry data stored in Grail.

It is fast, flexible, and designed for observability data at scale.

2 WHY ENGINEERS NEED QUERIES



- Go beyond dashboards
- Investigate unknown issues
- Correlate signals
- Build reusable insights
- Create custom metrics
- Automate investigations
- Answer "why did this happen?"

3 DQL BUILDING BLOCKS (QUERY PIPELINE)



FETCH DATA
from data sources



FILTER
refine the dataset



SELECT
choose fields



SORT
order the results



AGGREGATE
group and summarize



TIME RANGE
set the time window

4 CORE CLAUSES AT A GLANCE

- FETCH** | Get data
- FILTER** | Narrow results
- FIELDS** | Select columns
- SORT** | Order results
- SUMMARIZE** | Aggregate data
- LIMIT** | Limit rows
- TIMESERIES** | Time-based data

5 DQL CLAUSES EXPLAINED

FETCH DATA



Use data records, metrics, logs, traces, or events as your starting point.

Example:
fetch logs

FILTER



Use conditions to narrow down the data you care about.

Example:
| filter loglevel = "ERROR"

SELECT FIELDS



Pick only the fields you need in your results.

Example:
| fields timestamp, service.name, message

SORT



Order your results ascending (asc) or descending (desc).

Example:
| sort timestamp desc

AGGREGATE



Group and summarize data using functions like count(), avg(), sum(), etc.

Example:
| summarize count() by service.name

TIME RANGES



Always set the time window for meaningful results.

Example:
| timeframe = last 1h

6 BUILDING A SIMPLE INVESTIGATION QUERY (EXAMPLE)

QUESTION: Show me all ERROR logs from the checkout service in the last 1 hour, sorted by most recent.

- 1 FETCH DATA** - Start with logs
- 2 FILTER** - Keep only errors from a specific service
- 3 SELECT FIELDS** - Show relevant columns
- 4 SORT** - Show newest first
- 5 TIME RANGE** - Last 1 hour

```
1 fetch logs
2 | filter loglevel = "ERROR"
   and service.name = "checkout-service"
3 | fields timestamp, service.name, loglevel,
   message, trace.id
4 | sort timestamp desc
5 | timeframe = last 1h
   | limit 50
```

WHAT IT DOES

- Starts with all logs
- Keeps only ERROR logs from checkout-service
- Shows the most important fields
- Shows newest logs first
- Looks at the last 1 hour
- Limits to 50 rows for clarity

7 READING QUERY RESULTS (DON'T JUST COPY - UNDERSTAND!)

timestamp	service.name	loglevel	message	trace.id
2025-05-21 10:15:23	checkout-service	ERROR	Payment failed for orderId=9876	abc123def456
2025-05-21 10:15:20	checkout-service	ERROR	Timeout while calling payment gateway	abc123def456
2025-05-21 10:14:58	checkout-service	ERROR	Exception in PaymentClient	789ghi012jkl
2025-05-21 10:14:50	checkout-service	ERROR	HTTP 504 from payment-gateway	789ghi012jkl
2025-05-21 10:14:40	checkout-service	ERROR	Retry attempt 2 failed	345mno678pqr



WHAT TO LOOK FOR

- Patterns in messages (same error repeating?)
- Related trace.id (follow the trace!)
- Time gaps (when did it start?)
- Impact (how many errors? how often?)
- Correlate with metrics and other signals

8 DQL BEST PRACTICES



Start simple, then add more conditions.



Always set a time range.



Use fields to reduce noise.



Aggregate to see the big picture.



Correlate logs with traces and metrics.



Save useful queries for future investigations.



Be careful with sensitive data in logs and queries.



JUNIOR ENGINEER TIP

Great engineers ask better questions.
DQL helps you turn data into answers.

DASHBOARDS AND PRODUCTION VISUALIZATION

TURN DATA INTO CLEAR INSIGHTS, FASTER DECISIONS

1 WHAT MAKES A USEFUL DASHBOARD



- ✓ Shows system health at a glance
- ✓ Focuses on key signals, not everything
- ✓ Answers important questions quickly
- ✓ Helps you detect problems early
- ✓ Supports troubleshooting during incidents
- ✓ Gives context: who, what, when, impact
- ✓ Simple, clean and easy to read
- ✓ Updated in real time

2 KEY PRODUCTION VIEWS

INFRASTRUCTURE HEALTH



- CPU Usage
- Memory Usage
- Disk Usage
- Network Traffic
- Host Availability

APPLICATION HEALTH



- Request Rate
- Response Time
- Failure Rate
- Throughput
- User Experience

SERVICE-LEVEL SIGNALS



- Top Services
- Slowest Services
- Service Dependencies
- Endpoint Health
- Error Hotspots

ERROR TRENDS



- HTTP 4xx / 5xx
- Exception Rate
- Failed Requests
- Error Spikes
- Anomalies

DEPLOYMENT CONTEXT



- Recent Deployments
- Version Changes
- Traffic Impact
- Before vs After
- Related Problems

BUSINESS IMPACT



- Users Affected
- Critical Transactions
- Availability
- Experience Score
- Revenue Risk

3 CHOOSING USEFUL VISUALIZATIONS

GOAL	VISUALIZATION
Show trends over time →	 Time series chart
Compare values →	 Bar chart
See part vs whole →	 Pie / Donut chart
View distribution →	 Histogram
Check status →	 Status tiles
List top items →	 Top N table
Track deployments →	 Event timeline
Investigate details →	 Logs / DQL table

4 A SIMPLE OPERATIONAL DASHBOARD



5 AVOIDING DASHBOARD OVERLOAD

- ✗ Don't add every metric you see.
- ✗ Don't create dashboards that nobody uses.
- ✗ Don't make dashboards too busy.
- ✗ Don't show raw data without context.
- ✗ Don't build many dashboards with the same data.
- ✗ Don't ignore alerting. Dashboards inform, alerts demand action.

6 BUILDING AN OPERATIONAL DASHBOARD

- 1 Define the purpose (platform, app, service, business).
- 2 Identify 5-10 key metrics that matter.
- 3 Choose the right visualizations.
- 4 Add deployment and event context.
- 5 Keep it clean, simple and readable.
- 6 Share with the team and iterate.

7 JUNIOR ENGINEER TIP



A good dashboard does not just show data. It helps you answer:

- ✓ What is healthy?
- ✓ What is degrading?
- ✓ What changed?
- ✓ Where is the problem?
- ✓ What is the impact?
- ✓ What should I do next?



VISIBILITY WINS INCIDENTS. BUILD DASHBOARDS THAT DRIVE ACTION.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

PROBLEMS, EVENTS, AND DAVIS AI

AI-POWERED INTELLIGENCE. REAL-WORLD CLARITY.

1 WHAT DYNATRACE PROBLEMS REPRESENT



- Problems are Dynatrace's way of telling you that something important is wrong in your environment.
- They are impact-based, not just alert-based.
- Each Problem represents the impact on users, services, or infrastructure.
- Focus: What is the business impact?

2 EVENTS



- Events are changes in the environment.
- Examples:
 - Deployment started
 - New service discovered
 - Config change
 - Kubernetes pod restarted
- Events provide context around Problems.

3 ANOMALIES



- Anomalies are detected deviations from normal behavior.
- Dynatrace uses baselines and machine learning.
- Anomalies can trigger Problems if they cause an impact.
- Helps you detect issues early, even before users complain.

4 ENTITY RELATIONSHIPS



- Every entity is automatically related to others.
- Hosts, services, processes, containers, databases, and users.
- Relationships help Dynatrace understand impact and dependency.
- This fuels accurate root-cause analysis.

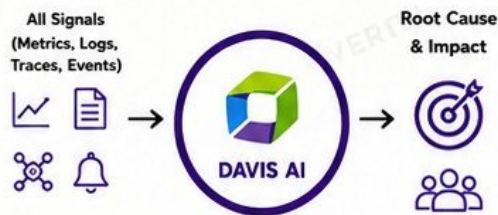
5 DAVIS AI



DAVIS®

- Davis AI is Dynatrace's artificial intelligence engine.
- Continuously analyzes metrics, logs, traces, topology, and behavior.
- Detects issues, predicts problems, and provides answers.
- Learns your environment automatically.
- Your AI-powered SRE assistant.

6 CAUSAL ANALYSIS & ROOT-CAUSE ASSISTANCE



- Davis AI analyzes millions of data points in real time.
- Determines what changed, what is impacted, and why.
- Shows the root cause and the impact path.
- Reduces Mean Time To Resolution (MTTR).

7 EVIDENCE vs AI CONCLUSIONS



EVIDENCE (FACTS)

- Actual data from your environment.
- Metrics, logs, traces, events.
- What really happened.
- Always verify the evidence.



AI CONCLUSIONS (INSIGHTS)

- Davis AI suggests probable causes.
- Based on patterns, baselines, and machine learning.
- Not always 100% accurate.
- Use AI to accelerate, but validate with evidence.



GOLDEN RULE

Trust, but verify.
AI guides you. You decide.

8 HOW AN ENGINEER INVESTIGATES A GENERATED PROBLEM



JUNIOR ENGINEER TIPS

- Don't start with dashboards. Start with the Problem.
- Always follow the data. Let evidence lead you.
- Use Davis AI to save time, not to skip thinking.

- Understand dependencies. The root cause is rarely where the pain is.
- Reduce noise by tuning alerts and focusing on real user impact.
- Observability is a skill. The more you investigate, the better you get.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

OBSERVE EVERYTHING. UNDERSTAND DEEPER. FIX FASTER.



ALERTING WITHOUT CREATING ALERT FATIGUE

RIGHT ALERTS. RIGHT PEOPLE. RIGHT TIME.

1 WHAT SHOULD TRIGGER AN ALERT?



- Something is wrong and needs attention now.
- Impacts users, services, or business.
- Breaches an SLO / SLA or risks it.
- Requires human action, not automation.
- When in doubt, don't alert. Inform with dashboards.

2 STATIC vs ADAPTIVE THRESHOLDS

STATIC THRESHOLDS



Fixed value. Good for stable, predictable systems.

ADAPTIVE THRESHOLDS



Learns normal behavior and adapts. Better for dynamic environments.

3 ANOMALY DETECTION



- Detects deviations from normal behavior.
- Uses machine learning baselines.
- Find issues you didn't know to watch.
- Reduce MTTR with early detection.

4 ALERT NOISE (THE ENEMY)



- Too many alerts desensitize teams.
- Causes missed critical alerts.
- Waste of time and mental energy.
- Focus on quality, not quantity.

5 PROBLEM NOTIFICATIONS



- Dynatrace groups events into Problems.
- You get one notification for many symptoms.
- Includes impact, root cause, and context.
- Alerts are about the Problem, not the noise.

6 OWNERSHIP AND ROUTING



- Every alert must have a clear owner.
- Route based on service, team, or business unit.
- Use escalation policies for unacknowledged alerts.
- Make it easy to reach the right person.

7 SEVERITY



SEV 1 - CRITICAL
Service down or major impact. Page immediately.



SEV 2 - HIGH
Major degradation. Urgent attention.



SEV 3 - MEDIUM
Minor impact or risk. Investigate soon.



SEV 4 - LOW / INFO
Informational or no user impact.

8 ACTIONABLE ALERTS



- Clear title and description.
- What is happening?
- What is the impact?
- What should I do?
- Include runbook / link.
- Provide enough context to act fast.

9 SYMPTOMS vs CAUSES

SYMPTOMS



What you see (Surface level)

- High CPU
- 500 errors
- Slow response
- Down service

≠

CAUSES



Why it happens (Root level)

- Database latency
- Memory leak
- Dependency failure
- Bad deployment

10

JUNIOR SRE RULE

DASHBOARDS INFORM, ALERTS DEMAND ACTION.

DASHBOARDS



- Observe and explore
- Understand trends
- Find patterns
- Inform decisions

ALERTS



- Something is wrong
- Needs immediate attention
- Take action
- Fix or mitigate

ALERTING BEST PRACTICES

- ✓ Start with user impact, not technology.
- ✓ Alert on problems, not on metrics alone.
- ✓ Use adaptive thresholds where possible.
- ✓ Review and tune alerts regularly.
- ✓ Silence responsibly during maintenance.
- ✓ Document runbooks for every alert.
- ✓ Continuously reduce noise.
- ✓ Keep learning from every incident.

REMEMBER



Good alerts save sleep.
Bad alerts destroy it.
Your future self will thank you.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BETTER ALERTS. FASTER RESPONSE. HAPPIER TEAMS.

KUBERNETES MONITORING WITH DYNATRACE

FULL VISIBILITY. FASTER INSIGHTS. HEALTHIER CLUSTERS.

1 CLUSTER VISIBILITY



- Overview of your entire Kubernetes environment.
- Real-time health of clusters.
- See capacity, workloads, services, and issues at a glance.
- Topology shows relationships across all entities.

2 NODES



- Monitor node health, CPU, memory, disk, and network.
- Detect saturation, pressure, and outages.
- See running pods and processes on each node.
- Node conditions and Kubernetes status.

3 NAMESPACES



- Isolate and monitor by namespace.
- Track resource usage, workloads, and events.
- Identify noisy neighbors and quota issues.
- Understand impact per namespace.

4 WORKLOADS



- Monitor Deployments, StatefulSets, DaemonSets, Jobs, and CronJobs.
- Track rollout status, availability, and restarts.
- See resource usage and performance.
- Link workloads to services and applications.

5 DEPLOYMENTS



- Monitor rollout progress and health.
- Detect failed rollouts and stalled updates.
- Correlate deployment changes with issues.
- View revision history and impact.

6 PODS



- Real-time pod status (Running, Pending, CrashLoopBackOff, etc.).
- Monitor restarts, IP, readiness, and age.
- See resource usage per pod.
- Drill down to containers inside each pod.

7 CONTAINERS



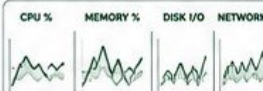
- Monitor CPU, memory, and I/O per container.
- Detect leaks, spikes, and throttling.
- See image, restarts, and exit codes.
- Correlate with logs and traces.

8 KUBERNETES EVENTS



- Capture all Kubernetes events in real time.
- Warnings, errors, scheduling issues, and failures.
- Correlate events with Problems and logs.
- Essential for fast root cause analysis.

9 RESOURCE UTILIZATION



- Monitor cluster, node, namespace, and pod usage.
- Detect overcommitment and pressure.
- Set baselines and thresholds.
- Understand requests vs limits vs usage.
- Plan capacity with confidence.

10 APPLICATION-TO-INFRASTRUCTURE CORRELATION



- Follow a request from the user down to the container, pod, node, and underlying infrastructure.
- Correlate metrics, logs, events, and traces.
- Instantly see how infrastructure issues impact applications.
- Faster root cause. Less guesswork.

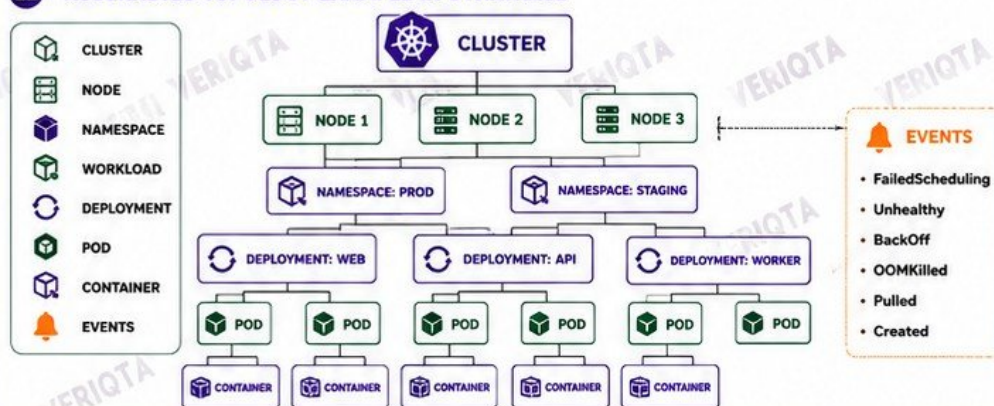
11 INVESTIGATING CRASHLOOPBACKOFF & RESOURCE PRESSURE



- CRASHLOOPBACKOFF**
- Check pod restart count and last state.
 - Inspect logs for errors.
 - Validate configuration and dependencies.
 - Correlate with events and deployments.

- RESOURCE PRESSURE**
- Check CPU / memory pressure on nodes.
 - Identify throttled or OOMKilled containers.
 - Review requests/limits and adjust.
 - Scale or optimize workloads.

12 KUBERNETES TOPOLOGY EXAMPLE IN DYNATRACE



13 BEST PRACTICES

- ✓ Monitor the golden signals for applications and workloads.
- ✓ Set meaningful alerts, not noise.
- ✓ Correlate across layers.
- ✓ Use dashboards for at-a-glance cluster health.
- ✓ Investigate with context: events, logs, metrics, traces.
- ✓ Review resource requests and limits regularly.
- ✓ Test failovers and upgrades with observability.



JUNIOR SRE TIP

You can't fix what you can't see. Monitor everything, correlate intelligently, and always start from the problem and follow the signals.



REMEMBER

Visibility is power.
Correlation is speed.
Context is everything.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

CLOUD AND PLATFORM OBSERVABILITY

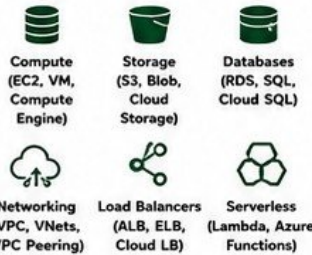
SEE EVERYTHING. CONNECT EVERY LAYER. SOLVE FASTER.

1 AWS, AZURE & GOOGLE CLOUD CONTEXT



- Dynatrace integrates with all major cloud providers.
- Collect metrics, logs, events, and trace data across cloud-native services.

2 CLOUD SERVICES



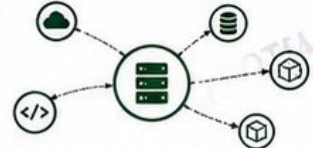
- Observe availability, latency, errors, throughput, and saturation.

3 KUBERNETES PLATFORMS



- Full visibility into clusters, nodes, pods, workloads, and namespaces.
- Correlate Kubernetes performance with cloud services.

4 INFRASTRUCTURE DEPENDENCIES



- Map and monitor dependencies across all layers.
- Know what your applications depend on.
- Understand impact before users do.

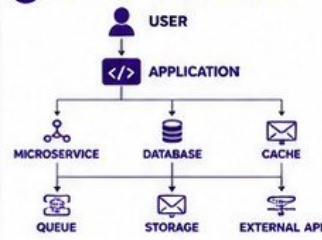
5 CLOUD METRICS

- CPU Utilization
- Memory Utilization
- Disk I/O
- Network Throughput
- Latency
- Error Rate
- Throttling
- Custom Metrics



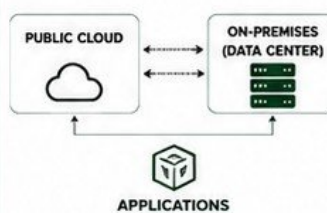
- Real-time and historical metrics from cloud services.
- Set baselines and detect anomalies.

6 SERVICE RELATIONSHIPS



- Visualize topologies automatically.
- See request flows and performance across all services.

7 HYBRID ENVIRONMENTS



- Dynatrace unifies visibility across clouds, on-prem, and edge.
- One platform. One view.

8 TAGS AND METADATA

- Environment (prod, dev, staging)
 - Team / Owner
 - Cost Center
 - Application
 - Business Unit
 - Version / Release
 - Custom Tags
- Use tags to filter, group and report.
- Better context. Faster troubleshooting.

9 FOLLOWING A PROBLEM ACROSS APPLICATION, PLATFORM, AND CLOUD LAYERS



THE OBSERVABILITY STACK WITH DYNATRACE



JUNIOR ENGINEER TIP

Always ask:
"What layer is showing the problem?"
Then move up or down the stack.



BEST PRACTICES

- Tag everything.
- Monitor dependencies, not just components.
- Correlate before you escalate.
- Automate baselines and anomaly detection.
- Keep dashboards focused and actionable.



REMEMBER

Observability is not about more data.
It's about the right data, in the right context, at the right time.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

OBSERVE EVERY LAYER. CORRELATE EVERYTHING. FIX FASTER.

SYNTHETIC MONITORING AND USER EXPERIENCE

PROTECT YOUR USERS. DETECT ISSUES BEFORE THEY NOTICE.

1 SYNTHETIC MONITORING



- Proactive monitoring from outside your stack.
- Simulates user actions on a schedule.
- Detects problems before real users are impacted.
- Available 24/7 from global locations.

2 BROWSER MONITORS



- Monitors real user journeys in a full browser.
- Validates UI, JavaScript, resources, and APIs.
- Capture page load time, errors, and screenshots.
- Perfect for critical user paths and transactions.

3 HTTP MONITORS



- Monitors URLs, APIs, and web services.
- Checks status codes, content, SSL, and headers.
- Lightweight and fast.
- Ideal for APIs, endpoints, and integration checks.

4 AVAILABILITY CHECKS



- Is the service up and reachable?
- Monitor from multiple locations.
- Detect downtime instantly.
- Get notified when a service becomes unavailable.

5 RESPONSE TIME



- Measure how fast your services respond.
- Track performance trends over time.
- Set thresholds for response time.
- Slow is the new down.

6 CRITICAL USER JOURNEYS



- Monitor the journeys that matter most to your business.
- Validate every step: UI, backend, APIs, and dependencies.
- Get alerts when any step fails or slows down.
- Ensure a great experience from start to finish.

7 REAL USER MONITORING (RUM)



- Collects real data from actual users.
- Captures page load, interactions, errors, and performance.
- Shows performance by device, browser, geography, and more.
- Understand the impact on real people.
- Complements synthetic monitoring.

8 SYNTHETIC vs REAL-USER DATA

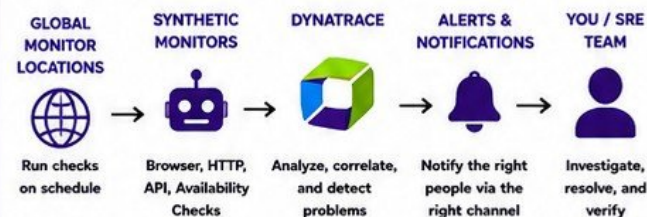
SYNTHETIC MONITORING	REAL-USER MONITORING
 Proactive (Outside-in)	 Real Experience (Inside-out)
 Global locations on schedule	 Real users, real conditions
 Consistent and repeatable	 Variable and unpredictable
 Detects issues before users	 Shows actual user impact
 Validates availability, functionality, performance	 Provides deep behavior insights

9 DETECTING PROBLEMS BEFORE CUSTOMERS REPORT THEM



- Synthetic monitors run 24/7.
- Fail fast and alert instantly.
- Identify issues before they affect revenue or reputation.
- Reduce MTTR and improve user satisfaction.
- Be proactive. Stay ahead.

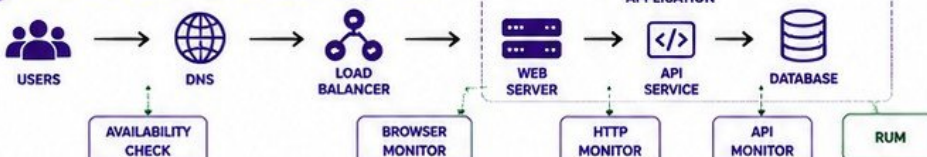
10 SYNTHETIC MONITORING ARCHITECTURE



11 BEST PRACTICES

- ✓ Monitor critical user journeys.
- ✓ Use multiple locations for availability.
- ✓ Set meaningful thresholds.
- ✓ Combine synthetic and RUM data.
- ✓ Alert on impact, not noise.
- ✓ Review and update monitors regularly.

12 EXAMPLE: E-COMMERCE SITE MONITORING



- ✓ Monitor the full path.
- ✓ Validate every layer.
- ✓ Detect issues early.
- ✓ Correlate synthetic + real-user data.
- ✓ Deliver a flawless experience.



JUNIOR SRE TIP

Synthetic monitoring tells you when something is wrong. RUM tells you how it affects real users. Use both. Trust the data. Protect the experience.



REMEMBER

The best incidents are the ones your users never notice because you detected and fixed them first.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SLOS, SLIS, AND PRODUCTION RELIABILITY

MEASURE WHAT MATTERS. MEET RELIABILITY TARGETS. EARN TRUST.

1 KEY CONCEPTS



SLI

Service Level Indicator
A measurable metric that reflects the health of your service.



SLO

Service Level Objective
A target value for your SLI over a specific time window.



SLA

Service Level Agreement
A commitment to your users or customers, often based on SLOs.



RELIABILITY

Delivering the right service with the right performance, consistently.

2 COMMON SLIS AND THEIR METRICS

AVAILABILITY



Measures uptime of your service.

Typical Metric:
(Successful Requests / Total Requests) x 100

Example SLI:
Availability %

LATENCY



Measures how fast your service responds.

Typical Metric:
Response Time (p50, p95, p99)

Example SLI:
p95 Latency

ERROR RATE



Measures requests that fail.

Typical Metric:
(Failed Requests / Total Requests) x 100

Example SLI:
Error Rate %

THROUGHPUT



Measures how much traffic your service handles.

Typical Metric:
Requests per Second (RPS)

Example SLI:
RPS

3 FROM SLI TO SLO: SETTING RELIABILITY TARGETS

SLI

What you measure
(e.g., Availability %)



SLO

What you target
(e.g., 99.9% over 30 days)



EXAMPLE SLO

Our checkout service availability SLO:
99.9% over 30 days (rolling window)

4 THE ERROR-BUDGET CONCEPT

If your SLO is 99.9% availability:



Error Budget
0.1%
(Max allowed failure)

You can "spend" this budget when things go wrong.

WHY ERROR BUDGETS MATTER

- ✓ Balance reliability and velocity
- ✓ Prevent overworking teams
- ✓ Encourage risk-aware decisions
- ✓ Drive conversations with data
- ✓ Protect user trust

5 MONITORING BUSINESS-CRITICAL SERVICES



IDENTIFY WHAT MATTERS

List your critical user journeys and services.



DEFINE SLIS AND SLOS

Pick the right metrics and set realistic targets.



MONITOR CONTINUOUSLY

Use real-time dashboards and alerts to track your SLIs.



DETECT AND RESPOND

Act quickly when SLIs drop below the SLO threshold.



LEARN AND IMPROVE

Reduce errors, improve performance, and raise the bar over time.

EXAMPLE BUSINESS-CRITICAL SERVICES

- ✓ Checkout Service
- ✓ Payment Processing
- ✓ User Authentication
- ✓ Order Management
- ✓ API Gateway
- ✓ Customer Support Portal

FOCUS ON USER IMPACT, NOT JUST SYSTEMS.

6 CONNECTING DYNATRACE WITH SRE PRACTICES

DYNATRACE HELPS YOU

- ✓ Capture accurate SLIs
- ✓ Visualize trends and performance
- ✓ Create SLO dashboards
- ✓ Set up alerts on SLO burn rates
- ✓ Analyze root causes with full context
- ✓ Prove reliability to stakeholders

SRE BENEFITS

- ✓ Data-driven reliability
- ✓ Reduce Mean Time to Detect (MTTD)
- ✓ Reduce Mean Time to Resolve (MTTR)
- ✓ Optimize error budget usage
- ✓ Deliver high-quality services consistently

7 SRE RELIABILITY FLYWHEEL



DEFINE
SLIS & SLOS



MONITOR &
MEASURE



ANALYZE &
UNDERSTAND



IMPROVE &
RELEASE



RELIABLE
SERVICE



HAPPY
USERS



CONTINUOUS
IMPROVEMENT



REMEMBER

Reliability is not a feature, it is a habit. Measure it. Improve it. Deliver it. Repeat.



JUNIOR ENGINEER TIP:

Start simple. Choose a few key SLIs, set realistic SLOs, and build from there. Reliability is a journey, not a one-time project.



PRINCIPLE:

You cannot improve what you do not measure.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DYNATRACE ADMINISTRATION, SECURITY, AND GOVERNANCE

SECURE ACCESS. ORGANIZE SMART. GOVERN WITH CONFIDENCE.

1 USERS AND GROUPS



- Create users for individuals.
- Use groups to manage access at scale.
- Assign permissions at the right level.
- Integrate with SSO/IdP for enterprise access.

2 ACCESS CONTROL



- Role-Based Access Control (RBAC).
- Environment-level permissions.
- Fine-grained permissions for apps, dashboards, and settings.
- Control who can view, edit, or administer.

3 LEAST PRIVILEGE



- Grant only the minimum access required.
- Avoid admin access for daily tasks.
- Review permissions regularly.
- Reduce blast radius of misuse.

4 ENVIRONMENT ORGANIZATION



- Use environments to separate Dev, Test, Staging, Prod.
- Keep data, users, and configurations isolated.
- Apply environment naming standards.
- Align environment setup with your delivery pipeline.

5 MANAGEMENT ZONES & SEGMENTATION



- Management zones group monitored entities.
- Segment by team, application, or business unit.
- Simplify access, dashboards, and alert routing.
- Prevent cross-team noise and complexity.

6 TAGS



- Use tags to add context to entities.
- Tag by application, team, environment, criticality.
- Enable smart filtering, dashboards, and alerts.
- Keep tag keys consistent.

7 NAMING CONVENTIONS



- Use clear, consistent names for environments, teams, and entities.
- Example: team-service-env
- Consistent naming improves search, filters, and automation.
- Document your standards.

8 API ACCESS AND TOKENS



- Use API tokens for automation and integrations.
- Assign the minimum scopes required.
- Rotate tokens regularly.
- Never hardcode tokens in scripts or repositories.

9 PROTECTING MONITORING CREDENTIALS



- Store tokens and credentials in a secure vault.
- Restrict who can create or view tokens.
- Monitor token usage and activity.
- Treat monitoring access like production access.

10 GOVERNANCE IN SHARED ENVIRONMENTS

- ✓ Define clear roles and responsibilities.
- ✓ Establish access request and review processes.
- ✓ Monitor configuration changes and user activity.
- ✓ Enforce policies for tags, naming, and alerting.
- ✓ Regularly audit access, tokens, and integrations.
- ✓ Document governance policies and exceptions.

GOVERNANCE PRINCIPLES



SECURITY FIRST
Protect data, users, and systems.



TRANSPARENCY
Everyone knows who has access and why.



ACCOUNTABILITY
Changes, access, and actions are traceable.

THE GOAL



Right people. Right access.
Right data. Right time.
Secure. Organized. Reliable.



JUNIOR ENGINEER TIP:

Good observability starts with good governance.
Clean access, clear organization, and strong security make everything else easier.



REMEMBER:

A well-governed Dynatrace environment is a reliable Dynatrace environment.



@veriqta



veriqta



veriqta



veriqta



@veriqta



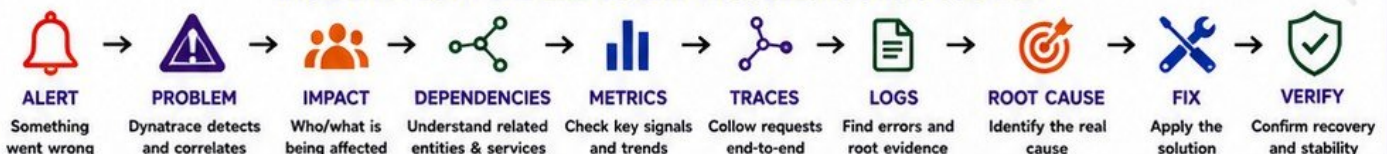
@veriqta

PRODUCTION INCIDENT TROUBLESHOOTING WITH DYNATRACE

A SYSTEMATIC APPROACH. FIND THE CAUSE. FIX IT FAST. VERIFY REAL RECOVERY.

1		START FROM THE SYMPTOM - Check alerts, user reports, or SLO violations. - Understand what is failing and for whom.	 DYNATRACE TIP Use Dashboards or Problems to see real-user and system impact.
2		CHECK THE PROBLEM - Open the Dynatrace Problem. - Review impact, root cause analysis, and affected entities.	 DYNATRACE TIP Davis AI gives you a head start, not the final answer.
3		IDENTIFY AFFECTED ENTITIES - Review impacted applications, services, hosts, and users. - Understand the scope and blast radius.	 DYNATRACE TIP Use Smartscape to see relationships and dependencies.
4		INSPECT RECENT CHANGES - Check deployments, config changes, releases, or incidents. - Correlate the timing with the issue.	 DYNATRACE TIP Use Deployments, Events, and Annotations.
5		CHECK SERVICE HEALTH - Review key metrics: rate, response time, failure rate, throughput. - Compare with historical baseline.	 DYNATRACE TIP Use Service Overview and Metrics Explorer.
6		FOLLOW TRACES - Open distributed traces for failing requests. - Find latency hotspots and error locations.	 DYNATRACE TIP Use PurePath to follow the request across services.
7		INSPECT LOGS - Search logs for errors, exceptions, and warnings. - Correlate logs with traces and timestamps.	 DYNATRACE TIP Use Log Viewer with filters and entity context.
8		CHECK INFRASTRUCTURE - Review host, container, and network health. - Look for resource saturation or failures.	 DYNATRACE TIP Use Infrastructure Overview and Kubernetes dashboards.
9		FORM AND TEST A HYPOTHESIS - Based on evidence, propose the likely root cause. - Validate by gathering more data.	 DYNATRACE TIP Avoid assumptions. Let data confirm your theory.
10		VERIFY RECOVERY - Confirm metrics return to normal. - Validate with SLOs, dashboards, and user experience.	 DYNATRACE TIP Recovery is complete when impact is gone and stable.

INCIDENT FLOW: THE END-TO-END TROUBLESHOOTING JOURNEY



JUNIOR ENGINEER TIP
 Stay calm. Follow the flow. Use Dynatrace to gather facts, not opinions. Good troubleshooting is 80% process, 20% tools.

REMEMBER
 Speed comes from a clear process. Accuracy comes from good data.

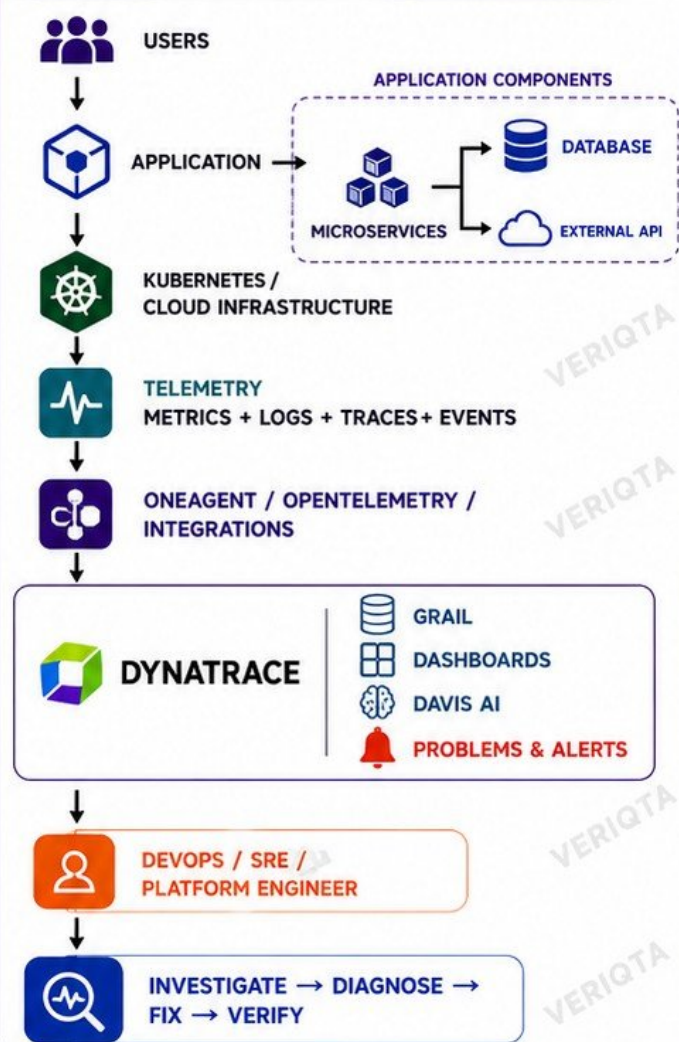
FINAL PRODUCTION LAB: INVESTIGATE A FAILING APPLICATION

PRACTICE. INVESTIGATE. SOLVE. VERIFY. DELIVER RELIABILITY.

LAB STEPS: FROM NORMAL TO RECOVERY

-  **MONITOR A KUBERNETES APPLICATION**
 - Deploy and connect Dynatrace to the app and cluster.
-  **ESTABLISH NORMAL BEHAVIOR**
 - Understand baseline metrics, traffic, and dependencies.
-  **CREATE AN OPERATIONAL DASHBOARD**
 - Build dashboards for infra, services, and business KPIs.
-  **OBSERVE REQUEST TRAFFIC**
 - Monitor live traffic, throughput, and user behavior.
-  **INSPECT INFRASTRUCTURE METRICS**
 - Check node, pod, CPU, memory, disk, and network.
-  **INTRODUCE APPLICATION LATENCY**
 - Simulate latency in a backend service.
-  **GENERATE FAILED REQUESTS**
 - Trigger errors using a failing endpoint or dependency.
-  **DETECT THE RESULTING PROBLEM**
 - Dynatrace detects anomalies and creates a Problem.
-  **IDENTIFY AFFECTED SERVICES**
 - Find impacted services, endpoints, and users.
-  **FOLLOW A DISTRIBUTED TRACE**
 - Trace failing requests end-to-end.
-  **CORRELATE LOGS**
 - Search logs to find errors and exceptions.
-  **INSPECT KUBERNETES WORKLOAD HEALTH**
 - Check pods, deployments, events, and resource usage.
-  **DETERMINE THE LIKELY ROOT CAUSE**
 - Use evidence from metrics, traces, logs, and infra.
-  **APPLY THE SIMULATED FIX**
 - Fix configuration, code, or dependency.
-  **VERIFY RECOVERY**
 - Confirm metrics, requests, and errors are back to normal.
-  **CONFIRM TELEMETRY RETURNED TO NORMAL**
 - Validate dashboards, SLOs, and system health.

FINAL ARCHITECTURE: OBSERVABILITY FLOW



PROGRESSION: PRODUCTION-FOCUSED OBSERVABILITY JOURNEY



JUNIOR ENGINEER TIP

- Observability is a skill built through practice.
- Always follow the data, not assumptions.
- Good engineers reduce MTTR with evidence, not guesswork.



REMEMBER

- Dashboards show you the picture.
- Telemetry gives you the story.
- You find the cause. You drive the fix.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta